MONOCHROME.TF

Comprehensive Feature & Technical Reference

An open-source, privacy-respecting, ad-free TIDAL Web UI

Official Instance: monochrome.tf

Source: github.com/monochrome-music/monochrome

License: Apache 2.0 | Language Stack: JavaScript, CSS, HTML

1. Overview

Monochrome is an open-source, privacy-respecting, advertisement-free web client for the TIDAL music catalog. Rather than running its own media servers, Monochrome acts as a sophisticated front-end layer that communicates with TIDAL's infrastructure through the Hi-Fi intermediary API, delivering streams directly to the user's browser. The result is a minimalist, clutter-free listening experience with feature parity—and in several dimensions, feature superiority—over the official TIDAL application.

The project is maintained by a small core team under the monochrome-music GitHub organization, with an active community of contributors and self-hosters. The primary live instance is served at monochrome.tf, with a secondary official mirror at monochrome.samidy.com and growing number of community-operated instances.

---

2. Architecture & Technology Stack

2.1 Frontend

Monochrome is a single-page application (SPA) built with vanilla JavaScript, CSS, and HTML — deliberately avoiding heavyweight frameworks in favour of a lightweight, fast-loading bundle. The codebase is organized under a /js directory for application logic and a /assets directory for static resources, with styles.css housing the global stylesheet and index.html as the entry point.

The application is served via Vite (a modern ES-module-based build tool and dev server) and distributed as a production build through Vite's preview mode. For self-hosters, the canonical deployment path is Docker Compose, which packages the Vite-served frontend into a container accessible on port 3000.

2.2 API Layer — Hi-Fi

The core data and streaming layer is provided by Hi-Fi, an open-source API project that proxies requests to TIDAL's backend. Monochrome ships with support for multiple Hi-Fi API endpoint instances simultaneously, with automatic failover if the primary instance becomes unavailable. The official API endpoints are:

- https://api.monochrome.tf — primary official endpoint
- https://monochrome-api.samidy.com — secondary official endpoint

Live uptime status for the API layer is exposed as machine-readable JSON at Cloudflare Worker URLs, allowing clients to programmatically check availability before making requests.

2.3 Backend Services

While the UI layer is stateless, Monochrome relies on two backend services for user account functionality:

Appwrite (Authentication)

Appwrite is used for the account system, handling email/password registration and Google OAuth sign-in. The team extensively customized Appwrite's standard modules to support SMTP delivery and to work around bugs in Appwrite's native Google OAuth implementation. Because these customizations introduce domain-locking (Appwrite's JWT issuance is restricted to authorized origins), account creation and login only work on officially sanctioned instances —

monochrome.tf and monochrome.samidy.com. Self-hosted deployments cannot use the account system as-is.

PocketBase (Library Sync)

PocketBase acts as the lightweight cloud database for syncing personal library data (favourites, playlists, listening history) across devices. Users may optionally configure custom PocketBase and Appwrite endpoint URLs in Settings for advanced or self-hosted setups.

2.4 Infrastructure

Component	Details
CDN / Proxy	Cloudflare CDN with Browser Insights enabled
TLS Certificate	Issued via WE1 authority; 3-month QUIC rotation
Domain Registrar	OVH (monochrome.tf)
Domain Verification	Google domain ownership verification active
Self-hosting	Docker Compose; exposes port 3000 via Vite preview
Tailscale Support	vite.config.js allowedHosts configuration for private mesh

2.5 Language Breakdown

Language	Approximate Share
JavaScript	67.2%
CSS	20.9%
HTML	11.9%

---

3. Audio Streaming & Playback

3.1 Audio Quality

Monochrome exposes TIDAL's full quality ladder through the Hi-Fi API, including Hi-Res FLAC — a lossless format that goes beyond Spotify Premium's 320 kbps Ogg Vorbis ceiling. Available quality tiers include:

- Hi-Res FLAC (up to 24-bit / 192 kHz master quality audio)
- FLAC (CD-quality lossless, 16-bit / 44.1 kHz)
- High AAC (~320 kbps)
- Normal AAC (~96 kbps)

Audio plays natively in the browser; no plug-ins or codecs need to be installed by the user.

3.2 Queue Management

The player includes a full queue system supporting:

- Manual track reordering within the queue

- Shuffle mode — randomizes playback order
- Repeat modes — repeat all, repeat one, or off
- Recently Played tracking for instant re-access to listening history

3.3 Media Session API

Monochrome integrates the browser's Media Session API, allowing the operating system's native media controls (taskbar, lock screen, headphone buttons, keyboard media keys) to control playback — play, pause, skip, previous, and seek.

3.4 Audio Visualizer

An optional audio visualizer renders animated graphics synchronized to the playing track. Two tunable parameters are exposed in Settings:

- Visualizer Sensitivity — controls the intensity of the effect; accompanied by a photosensitive epilepsy warning for high values
- Visualizer Brightness — independent brightness control

3.5 Track Downloads

Monochrome supports downloading individual tracks, full albums, and playlists as audio files. Metadata (artist, album, artwork, track number, etc.) is automatically embedded into the downloaded files. Additional download options include:

- Force ZIP as Blob — downloads ZIP archives in memory rather than streaming to disk (useful for environments where streaming ZIP causes browser issues)
- Download Lyrics — optionally bundle .lrc lyric files alongside downloaded audio

---

4. Content Discovery & Browsing

4.1 Search

Full-text search across TIDAL's catalog covers tracks, albums, artists, and playlists. Results are surfaced in real time via the Hi-Fi API.

4.2 Explore / Hot & New

A dedicated Explore page aggregates:

- Hot & New — recently added music trending globally
- Genre-specific trending — filtered views for individual genres

This surface provides passive discovery without requiring the user to know what they are looking for.

4.3 Smart Recommendations

For any track, album, or artist, Monochrome can surface recommended similar content powered by TIDAL's recommendation engine via the Hi-Fi API.

4.4 ArtistGrid — Unreleased Music

Monochrome integrates with ArtistGrid, a platform that hosts unreleased and exclusive tracks. This provides access to music not yet available on any mainstream streaming service.

---

5. Personal Library & Playlists

5.1 Library

Authenticated users have a comprehensive personal library for storing favourite tracks, albums, and artists. The library syncs across devices via PocketBase cloud storage. Library data can be exported and imported as JSON through the Backup & Restore feature in Settings.

5.2 Playlist Management

Users can create, edit, and delete personal playlists. Playlists support a public sharing toggle, making them accessible to anyone with the link without requiring an account.

5.3 Playlist Import

Monochrome supports importing playlists from external platforms — including Spotify and YouTube Music. The system performs a best-effort song-matching search against TIDAL's catalog. The import feature carries a built-in warning that the match quality is imperfect; users are advised to review imported playlists and manually remove incorrect additions. Unmatched songs are reported after import.

---

6. Lyrics

6.1 Synchronized Lyrics

Monochrome supports time-synchronized lyrics that scroll in real time with the playing track, sourced natively from TIDAL's catalog where available.

6.2 Karaoke Mode

Karaoke mode highlights the current lyric line, providing an interactive sing-along experience on top of the synchronized display.

6.3 Genius Integration

For tracks not covered by TIDAL's native lyrics, Monochrome falls back to the Genius lyrics database, significantly broadening lyric availability.

6.4 Romaji Conversion

Japanese-language lyrics can be automatically transliterated into Romaji (Latin characters) — a feature useful for learners or listeners who prefer phonetic reading over kanji/kana.

---

7. Accounts & Cross-Device Sync

7.1 Authentication

Accounts are available exclusively on the official instances. Two sign-in methods are supported:

- Google OAuth — one-click sign-in via a Google account
- Email / Password — traditional registration with SMTP-verified email

The team emphasizes a minimal data collection philosophy: only music library data and a randomized internal identifier are stored. No email addresses, usernames, or personally identifiable information are retained in the user-facing database.

7.2 Cross-Device Sync

Once authenticated, the following data is synced via PocketBase across all signed-in devices:

- Favourited tracks, albums, and artists
- Playlists and their contents
- Listening history

7.3 Backup & Restore

Users can export their full library state — playlists, favourites, history — as a JSON file, and restore from that file on any device or instance. This provides a device-independent backup mechanism independent of cloud sync.

7.4 Data Management

Settings expose granular controls for data management:

- Reset Local Data — clears all localStorage and browser-cached data without affecting cloud sync
- Clear Cloud Data — permanently deletes all user data from the cloud (irreversible)

---

8. Scrobbling & Social Integrations

8.1 Last.fm

Last.fm scrobbling is natively supported. Authorization is performed via Last.fm OAuth and is stored locally in the browser — meaning it must be re-authorized on each new device. When connected, Last.fm also surfaces the user's recent activity and top stats on their Monochrome profile page.

8.2 ListenBrainz

ListenBrainz — the open-source, privacy-respecting alternative to Last.fm maintained by the MetaBrainz Foundation — is also supported for scrobbling listens. This gives users a fully open-source end-to-end listening audit trail.

8.3 Discord Rich Presence

Dynamic Discord embeds allow Monochrome to post now-playing information to a user's Discord status, showing track, artist, and album artwork in real time.

---

9. User Interface & Customization

9.1 Design Philosophy

The UI is minimalist and deliberately monochromatic, stripping away the visual noise common in commercial streaming applications. Album art provides the primary colour accent; the surrounding chrome stays neutral.

9.2 Now Playing View

The Now Playing panel is configurable: users can choose what is displayed when clicking album art — for example, a full-screen art view, lyrics panel, queue, or visualizer.

9.3 Theme System

Monochrome has a community theme system. Users who are logged in can upload custom themes; anyone can browse and apply uploaded themes. Themes modify the application's CSS variables to change colours, typography, and visual feel without requiring a code fork.

9.4 Keyboard Shortcuts

A comprehensive set of keyboard shortcuts covers all major playback and navigation actions. Shortcuts are fully rebindable: clicking any shortcut entry in the Keyboard Shortcuts panel puts it into capture mode, and pressing the new key combination saves the binding. Default bindings follow standard media player conventions.

9.5 Command Palette

A command palette (typically invoked via a keyboard shortcut) provides a quick-search interface for navigating to any section, triggering actions, or finding content — a power-user affordance inspired by code editors like VS Code.

9.6 Navigation Behaviour

Two navigation-related settings control modal/panel interaction with the browser's back button:

- Close Modals on Navigation — automatically dismisses open modals (lyrics, queue, etc.) when navigating to a new page
- Intercept Back to Close Modals — repurposes the browser's back gesture to close open modals first; a second back press navigates away

---

10. Progressive Web App (PWA) & Offline Support

Monochrome ships with a full PWA manifest (manifest.json) and service worker (sw.js), enabling installation as a first-class application on all major platforms — Windows, macOS, Linux, Android, and iOS — directly from the browser without an app store.

PWA capabilities include:

- Install to home screen / taskbar
- Offline caching of the application shell and previously loaded UI assets
- App-like interaction patterns (full-screen, no browser chrome)
- Media Session API integration (system-level now-playing controls)

The Auto-Update setting causes the app to automatically reload when a new version of the service worker is detected, keeping users on the latest build without manual intervention.

An API response cache stores recent search and catalog responses locally, reducing network requests and improving load times on repeat visits. The cache can be manually cleared from Settings.

---

11. Desktop & Mobile Applications

11.1 Desktop App

A dedicated desktop application (monochrome-music/desktop-app) wraps the web UI in a native shell. A built-in update checker allows the desktop application to notify users of new releases and trigger updates from within the app.

11.2 Mobile App

A separate native mobile application (monochrome-music/mobile-app) is under active development (most recently updated March 2026), providing a dedicated Android and/or iOS experience beyond the PWA.

11.3 Legacy Instance

A monochrome-legacy repository, maintained by community contributor @binimum, provides a version of the application compatible with older hardware and browsers that cannot run the current build's JavaScript requirements.

---

12. Privacy & Data Practices

Privacy is a first-class design goal for Monochrome. The key commitments, as stated by the project, are:

- No advertisements of any kind
- No sale of user data
- Minimal data collection — only music library data and a randomized anonymous ID are stored; no emails, usernames, or sensitive information
- No hosting or storage of media files — all audio streams are delivered directly from TIDAL's infrastructure
- Optional analytics — the application can send anonymous usage telemetry, but this is user-controllable from Settings

The application carries a standard legal disclaimer clarifying that it does not control, operate, or maintain any external content sources, and that users are responsible for compliance with applicable laws in their jurisdiction.

13. Settings Reference

The Settings panel is organized into functional sections. The following table summarizes all known configurable options:

Setting	Description
Keyboard Shortcuts	View and rebind all keyboard shortcuts via interactive capture
Cache	Stores API responses locally to reduce network requests; clearable on demand
Auto-Update App	Automatically reloads when a new service worker version is detected
Desktop Update	Checks for updates to the desktop application build
Analytics	Toggles sending anonymous usage data to the development team
Reset Local Data	Clears all localStorage and browser cache without affecting cloud sync
Clear Cloud Data	Permanently deletes all cloud-stored data (irreversible)
Backup & Restore	Export or import library and history as a JSON file
Scrobbling (Last.fm)	Authorize Last.fm; stored locally per device
Scrobbling (ListenBrainz)	Authorize ListenBrainz for open-source listen tracking
Visualizer Sensitivity	Controls intensity of audio visualizer (epilepsy warning at high values)
Visualizer Brightness	Controls brightness of the visualizer overlay
Now Playing View Mode	Choose what displays when clicking album art
Close Modals on Navigation	Dismiss open modals when navigating to a new page
Intercept Back to Close Modals	Back gesture closes modals before navigating away
Force ZIP as Blob	Download ZIP in memory instead of streaming to disk
Download Lyrics	Include .lrc files when downloading tracks/albums
Romaji Lyrics	Transliterate Japanese lyrics to Romaji (Latin characters)
Custom API Instance	Configure custom PocketBase / Appwrite endpoints
Upload Theme	Upload a custom CSS theme (requires account)

14. Instances & Deployment

14.1 Official Instances

URL	Role
monochrome.tf	Primary official instance
monochrome.samidy.com	Secondary official mirror

14.2 Community Instances

Community members may self-host the application via Docker Compose. A curated INSTANCES.md file in the repository lists known public community instances (e.g., mono.squid.wtf). Community instances have varying uptime, and account features are not available on self-hosted deployments due to Appwrite domain restrictions.

14.3 Self-Hosting Steps

The canonical self-hosting procedure is:

- git clone https://github.com/monochrome-music/monochrome.git
- cd monochrome
- docker compose up -d
- Access the application at http://localhost:3000

For Tailscale mesh access, the vite.config.js preview.allowedHosts array should be updated with the Tailscale hostname, and preview.host should be set to true.

---

15. Feature Summary Matrix

Feature	Status / Notes
Hi-Res FLAC Streaming	Available; sourced from TIDAL via Hi-Fi API
No Advertisements	By design; no ad network integration
Progressive Web App	Full manifest + service worker; installable on all platforms
Offline Mode	Application shell cached via service worker
Media Session API	OS-level playback controls
Queue + Shuffle + Repeat	Full queue management
Recently Played History	Tracked locally and in cloud when signed in
Lyrics (Synchronized)	TIDAL native; karaoke mode available
Lyrics (Genius fallback)	Broadens coverage to non-TIDAL-licensed lyrics
Romaji Transliteration	Japanese lyrics → Latin characters
Playlist Import	Spotify, YouTube Music; best-effort matching

Feature	Status / Notes
Public Playlists	Shareable via link; no account required to view
Track / Album Downloads	With embedded metadata; optional .lrc files
Account System	Email + Google OAuth; official instances only
Cross-Device Sync	PocketBase cloud; library, playlists, history
Backup & Restore	JSON export/import
Last.fm Scrobbling	OAuth; per-device authorization
ListenBrainz Scrobbling	Open-source listen tracking
Discord Rich Presence	Dynamic now-playing embeds
Unreleased Music	ArtistGrid integration
Smart Recommendations	Powered by TIDAL recommendation engine
Explore / Hot & New	Genre and global trending pages
Audio Visualizer	Sensitivity + brightness adjustable
Keyboard Shortcuts	Fully rebindable
Command Palette	Quick search / navigation
Community Themes	Upload and apply CSS themes
Multiple API Instances	With automatic failover
Desktop App	Separate Electron-style wrapper; self-updating
Mobile App	Native app under active development
Legacy Mode	Separate branch for older hardware/browsers
Docker Self-Hosting	docker compose up -d; port 3000
Tailscale Support	vite.config.js allowedHosts configuration
Open Source	Apache 2.0; github.com/monochrome-music/monochrome

16. Legal & Sustainability Considerations

Monochrome occupies a legally ambiguous position. It does not host or distribute audio files, and describes itself as a web UI that relies on publicly accessible APIs. However, its dependence on the Hi-Fi API — which routes requests through TIDAL's infrastructure without a paid TIDAL subscription — raises questions about compliance with TIDAL's terms of service.

The project draws historical parallels to Grooveshark, a music streaming service that operated in a similar grey zone before a 2015 legal settlement forced it offline. The Monochrome team acknowledges this and the project's sustainability depends on:

- Continued operation of the Hi-Fi API without legal intervention from TIDAL or its rights-holder partners
- The ability of the development team to respond to authentication or protocol changes that TIDAL may implement
- Community engagement: the project accepts donations via Ko-fi (ko-fi.com/monochromemusic)

Users should be aware that libraries, playlists, and accounts could be disrupted if the underlying API changes or the project faces legal action. The Backup & Restore feature is the recommended mitigation for this risk.

— End of Document —